REVIO STOCKSENSE

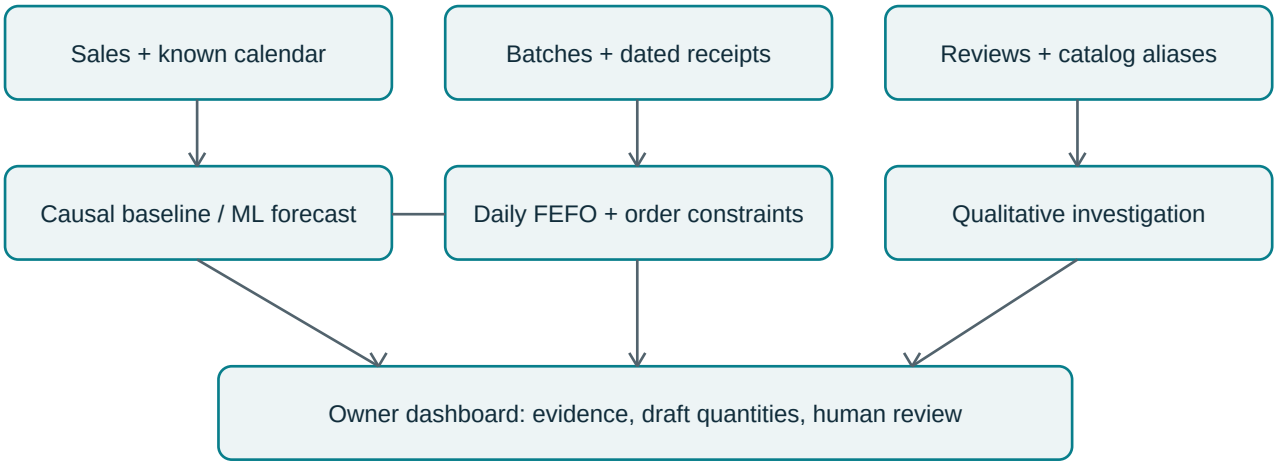
Technical Approach Report

Data, causal features, chronological evaluation and inventory mathematics.

17 October 2026
Independent proposal for Cynbit AI/ML Engineer Internship

Research claims, simulated measurements and future proposals are distinguished throughout. No live Revio integration or real-store ROI is claimed.

System at a glance



Implementation and evaluation snapshot: 17 October 2026. All bundled business records are synthetic. No API keys, LLM service or live Revio connection is used.

Problem and functional requirements

Estimate future product demand and translate it into reviewable inventory decisions. The system must distinguish observed sales from latent demand, use recorded batch dates, account for timed supply and refuse unsupported exact quantities. It supports sales analysis, 7/14/30-day forecasts, replenishment, seasonal/event evidence, expiry, slow movers, review intents and CSV/SQLite data management.

Nonfunctional goals are reproducibility, modularity, explicit failure states and understandable explanations. This is a single-user local proof of concept, not a production transaction system. Upload limits bound memory and training work; they are not performance guarantees.

Architecture

```
Six CSV tables -> atomic validation -> snapshot
  sales + calendar -> causal features -> baseline/ML comparison -> forecasts
  batches + orders + forecasts -> daily FEFO -> constrained purchasing review
  reviews + aliases -> qualitative intent/evidence -> investigation queue
  all outputs -> seven-section Streamlit dashboard -> CSV exports
  explicit save/load <-> local SQLite
```

Reviews intentionally bypass numerical forecasting. There is no training label that could justify converting a review count into units of demand. All outputs remain decision support.

Technology choices

Python 3.12, pandas/NumPy for tables and arrays, scikit-learn PoissonRegressor and HistGradientBoostingRegressor for demand, Streamlit and Plotly for the interface, SQLite from the standard library for persistence, pytest for checks and ReportLab for PDFs. Versions are pinned in requirements files. No deep learning framework is needed for this small tabular dataset. No prompt engineering is involved because no language model is called.

Dataset design and validation

Seed42 generates 12,240 daily sales rows covering 1,020 days, twelve stocked SKUs and one no-history SKU. Categories include beverages, dairy, snacks, pantry, gifting and essentials. Demand combines product base rate, smooth annual seasonality, weekday effects, modest trend, illustrative event effects and stochastic variation. Receipts cap actual sales; supply interruptions create censored observations. A final snapshot includes 24 batches, three purchase orders and fourteen reviews.

Six linked tables are validated together. Required columns, numeric finiteness/nonnegativity, dates, unique keys, product references, stock balance and order chronology are checked. Missing manufacturing/expiry is allowed but represented as unknown. Duplicate sale dates per SKU are rejected rather than aggregated silently. Availability fields are optional; absent fields prevent reliable lost-demand identification. See docs/Data_Card.md and src/data.py for the full schema.

The simulation stores latent demand only for a separate evaluation diagnostic. It never enters model features. The public Google Holidays in India calendar supplies 28 selected dates for 2024-2027, imported on 17 October 2026. Synthetic event effects were regenerated to match these dates; uploaded sales are never rewritten. This snapshot is not continuous OAuth synchronization. Regional variation and tentative dates remain relevant.

Censoring and historical analysis

`demand_series` masks known stockouts and days whose sales exhaust recorded availability, then reindexes missing dates to NaN. It does not label those days zero demand. The conservative sold-out rule can also mask a day whose true demand exactly equals stock; this trades lost observations for avoiding false certainty. Selection on available days can still bias learning downward.

Historical summaries retain recorded total units/revenue, recent 28-day observed mean, 7/30-day equivalents, previous-window growth and standard deviation. Zero-heavy history is marked intermittent; recent mean >=15 is fast moving and <3 slow moving. These thresholds are transparent demo heuristics, not learned business categories.

Selected seasonal model

One Poisson regression per product jointly learns trend, two annual sine/cosine harmonics, weekday indicators and separate festival preparation-window indicators (seven days before through event day). StandardScaler and PoissonRegressor(alpha=.03, max_iter=500) yield nonnegative expected counts through a log link [3]. Smooth annual terms avoid month-boundary steps [4]. Seasonal labels in reporting are broad display groups, not weather forecasts. There is no separate seasonal prediction or universal festival multiplier.

Training uses only uncensored observations before the cutoff. At least 180 observed days spanning 365 days are required; otherwise the model explicitly falls back to the 28-day average. No-history products remain unsupported. The model can miss sudden changes, promotions or regional differences; two or three annual cycles do not establish real-world reliability.

Legacy comparator feature engineering

For each SKU and forecast origin, construct lag1, lag7, mean7/28/84, std28, mean28-minus-mean84 trend and missing fraction over28 days. Add categorical product identity, horizon1-30, target weekday, sine/cosine of target month and whether an event is within the next seven days. Month encoding wraps December into January smoothly. Calendar dates are assumed known at the origin.

Historical target-day sales, future availability and latent simulated demand are excluded. Category is not separately encoded: product identity and history carry this small closed catalog. Future promotions are not supplied, so their effect remains an omitted driver. New SKUs cannot inherit a precise forecast from identity alone.

Comparator algorithms and training

Moving average repeats the last28-day observed mean. Weekly seasonal naive repeats the last seven calendar positions, filling missing positions with the past28-day mean. These are meaningful simple comparators.

The global direct model learns all products and horizons together: prediction = f(origin-history features, known future calendar, SKU, horizon). It uses 90 gradient-boosted tree iterations, 15 leaf nodes per tree, learning rate .08, L2=2, seed42 and squared-error loss. Each tree corrects residual errors from the current ensemble. Product index is categorical, not treated as a meaningful numerical order. Negative outputs are clipped to zero.

Weekly training origins start after83 calendar days. Targets extend at most30 days and never beyond the training cutoff. Only observed, uncensored targets are fitted. No internal random holdout is used: early stopping is disabled. Hyperparameters are fixed for this experiment rather than optimized on the final test set. If insufficient training samples exist, the UI falls back to the moving average.

Forecast eligibility requires at least14 observed historical days and seven observed values in the latest28 days. Unsupported products get missing forecasts and zero draft purchase quantity with an investigation message. The data-support label becomes limited if fewer than56 observations exist in84 days or more than20% of recent28 days are missing. It is not a probability or accuracy interval.

Chronological evaluation protocol

Four chronological validation windows select Seasonal Poisson regression by pooled MAE. Twelve subsequent disjoint 30-day rolling test windows cover 360 days. Each origin trains only on earlier data; the model choice stays frozen. Results are synthetic uncensored observations, not merchant performance.

Method	Validation MAE	Test MAE	Test RMSE	Test WAPE	Test bias
Seasonal Poisson regression	3.088	2.884	3.958	18.673%	-0.052
HistGradientBoosting	3.117	2.990	4.105	19.353%	-0.284
Moving average 28d	3.439	3.389	4.750	21.938%	-0.047
Weekly seasonal naive	4.213	4.240	5.852	27.446%	-0.236

Test count: 4,201 observed product-days. The selected model reduces MAE by 3.5% versus boosting and 14.9% versus the average on the same revised data. All four broad seasonal groups improve; this does not imply every SKU improves. Separate latent simulation diagnostic: n4,320, MAE2.925. WAPE is not classification accuracy. Old v1 results use different data and a shorter protocol and are archived in evaluation/archive/v1; they are not a direct before/after comparison.

MAE measures average absolute error; RMSE emphasizes larger errors; bias is prediction minus actual. Shorter uploaded histories use a restricted protocol and cannot establish full-year accuracy. The calendar was imported retrospectively, not from a historical point-in-time feed.

Seasonal and festival logic

The selected model learns product-specific season and event associations jointly. Forecasting contains one prediction chart with calendar markers; historical monthly/event evidence is expandable on the same page. The seasonal chart aggregates observed daily units by month across years. The event evidence table compares seven pre-event days with the preceding28-day baseline. It requires at least five observed event days,20 baseline days and two comparable occurrences. Overlapping events, promotions and trends confound this descriptive comparison. It never adds an extra uplift to model output. Missing sufficient evidence is reported explicitly.

Reorder mathematics and safety policy

Protection period = supplier lead time + review period. Target = sum of forecast over that period + safety stock. Safety stock = configured safety days times mean forecast over protection. Raw need = max(0, target - eligible on-hand - confirmed receipts + simulated expiry loss + backorders).

The default review period is7 days and safety buffer1.5 days. These are editable policy assumptions, not optimized cost or guaranteed service levels. Example:20/day for11 days +30 safety -80 usable -20 incoming =150 units. The unit test verifies this exact result.

Only confirmed purchase orders arriving after snapshot and within the relevant period count. Draft/cancelled/received records and overdue unconfirmed receipts do not count as future supply. Daily simulation flags shortages before new supply can arrive; a later order cannot repair a past lost sale. Protection longer than30 days withholds automatic quantities. Lead0 is conservatively next-day arrival because this is a daily snapshot model.

FEFO expiry and constraints

At each future day, add confirmed arrivals, expire batches whose date is before that day, then allocate demand to the earliest eligible expiry. Future recorded expiry dates are inclusive. Stock expiring on the snapshot date is excluded because forecasts start tomorrow. Missing expiry is not assumed safe; it blocks automatic purchasing pending verification. Expired stock is never sold.

For a batch80 units expiring in five future days at demand6/day, estimated sales are30 and unsold-at-expiry50. Multiple batches share demand under FEFO, avoiding double allocation. A batch expiring beyond the forecast horizon has unknown unsold-at-expiry, not zero risk. Recorded dates are never predicted by ML.

New orders are capped by residual forecast shortage during assumed remaining receipt life and available unit capacity at arrival before consumption. Then apply MOQ and pack rounding; if upward rounding violates the cap, round down to a feasible pack quantity or withhold if below MOQ. The system deliberately allows constrained quantities below the policy target and explains why. New receipts use catalog remaining-life assumptions until supplier batch labels exist. No monetary budget or simultaneous supplier optimization is implemented.

Slow stock and priorities

Excess = max(0, eligible on-hand -30-day forecast). Days cover = eligible quantity / mean forecast, a rough screening metric that does not account for future expiry. Slow-stock review also flags recent mean<3 or growth below-20%. It suggests purchasing review, condition inspection and permitted supplier return/promotion consideration, not automatic discounting or disposal.

Ordinal priorities: P1 for recorded expiry concerns or shortage before lead time; P2 for replenishment, constraints or unsupported history; P3 for excess; P4 for sufficient stock. Forecast support is a separate column. No arbitrary weighted score mixes incompatible units.

Review NLP

Aliases are matched longest first with nonoverlapping spans, so sugar-free cola does not incorrectly match ordinary cola inside the same phrase. English regex rules detect availability complaints, requests, variants and quality complaints; limited negation exceptions handle phrases such as not out of stock. Supplied product IDs are treated as input assertions. Evidence text and original review remain visible. Rules are not a trained sentiment classifier and there is no claimed NLP accuracy benchmark. Multi-product attribution, sarcasm and multilingual reviews remain weaknesses.

UI, persistence and privacy

Streamlit offers seven sections and caches computation by snapshot content. Demo metrics apply only to the demo; uploaded/local-loaded snapshots start with the baseline until their own evaluation is run. All six uploads replace state only after validation succeeds. Review-only replacement also validates the related snapshot. SQLite save/load is explicit and uses fixed table names and atomic file replacement. Formula-leading strings are escaped in supplied CSV export buttons.

Data stays in the local process and optional `.local/snapshot.sqlite`; it is not sent to a model API. The application has no authentication and should bind to127.0.0.1. Production requires tenant isolation, encryption, access control, retention/deletion policy, audit logs and secure connectors. Streamlit's built-in grid exports are not the application-controlled safe CSV export path; use the named download buttons for untrusted text. No secrets belong in Git.

Verification and deployment plan

Executed pytest unit/integration tests cover schema failures, stock ledger, SQLite roundtrip,150-unit reorder,50-unit expiry example, FEFO allocation, unknown/expired exclusion, receipt timing, causal feature invariance, no-history forecasts, overstock, horizon suppression, festivals, review negation and seven-section UI navigation. Actual browser inspection checks chart rendering after restarting the old server to clear stale modules.

Run locally with Python3.12 and requirements.txt. The first forecast fit is cached per snapshot. The prototype bounds100 products,100,000 rows/table and2,000 history days; no load-test SLA is claimed. A real deployment should move training to scheduled jobs, version datasets/models, separate tenant data, monitor drift and prediction coverage, and keep human purchasing approval.

Limitations and next experiments

Synthetic single-store data, pooled model selection, uncensored selection bias, uncertain incoming life, simplified lost-sales/backorder treatment, retrospectively imported public calendar and English-only rules limit generalization. Training targets overlap across historical origins; this is direct forecasting training, while chronological validation windows remain disjoint. No calibrated uncertainty interval, service-level guarantee, validated ROI or real integration is implemented.

Next: consented real data, per-SKU residual analysis, robust censor-aware demand models, intermittent-demand comparators, promotion schedules, supplier uncertainty, policy-cost simulation and controlled pilots. Confirm original assessment timezone independently; no official timezone evidence was available.

References

[1] Hyndman and Athanasopoulos, Forecasting: Principles and Practice, forecasting toolbox and time-series cross-validation, https://otexts.com/fpp3/toolbox.html (accessed2026-10-17).

[3] scikit-learn PoissonRegressor API, https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.PoissonRegressor.html (accessed2026-10-17).

[4] Hyndman and Athanasopoulos, dynamic regression and harmonic terms, https://otexts.com/fpp3/dynamic-regression.html (accessed2026-10-17).

[2] scikit-learn HistGradientBoostingRegressor API, https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.HistGradientBoostingRegressor.html (implementation reference; installed version is pinned).